AEGES-Q: Post-Quantum Cryptography Benchmark

1. Research Objectives and Experimental Design

1.1 Research Context

AEGES-Q investigates the integration of classical and post-quantum cryptographic mechanisms into an adaptive network security system. This experiment evaluates **ML-KEM-768**, a standardized post-quantum key-encapsulation mechanism, and compares its computational characteristics with the project's existing classical key-establishment baseline based on **X25519 and HKDF-SHA256**.

The experiment extends the existing classical cryptography work without modifying its implementation. The objective is to establish a reproducible experimental baseline for assessing the feasibility of incorporating post-quantum key establishment into AEGES-Q.

1.2 Research Objectives

The experiment addresses the following objectives:

1. **Functional correctness:** Verify that ML-KEM-768 key generation, encapsulation, and decapsulation operate correctly using a standards-compliant implementation.
2. **Performance characterization:** Measure the execution time of key generation, encapsulation, and decapsulation independently.
3. **Communication overhead:** Record the sizes of public keys, private keys, encapsulation ciphertexts, and derived shared secrets.
4. **Classical comparison:** Compare ML-KEM-768 with the existing X25519 + HKDF-SHA256 baseline, accounting for the differences between their cryptographic interfaces.
5. **Reproducibility:** Record software versions, experimental parameters, randomization policies, and measurement procedures so that the results can be independently reproduced.
6. **Integration relevance:** Assess the performance and data-size implications of introducing post-quantum key establishment into the AEGES-Q architecture.

1.3 Research Questions

This experiment investigates four research questions:

- **RQ1:** What are the measured computational costs of ML-KEM-768 key generation, encapsulation, and decapsulation on the experimental system?
- **RQ2:** How do the public-key, private-key, ciphertext, and shared-secret sizes compare with the corresponding classical key-establishment data?

- **RQ3:** How does ML-KEM-768's measured performance compare with the existing X25519 + HKDF-SHA256 implementation under a consistent benchmarking environment?
- **RQ4:** What performance and implementation considerations affect the suitability of ML-KEM-768 for integration into AEGES-Q?

1.4 Scope and Experimental Boundaries

The primary post-quantum algorithm under evaluation is **ML-KEM-768**, standardized in NIST FIPS 203. The classical reference mechanism is X25519 for key agreement, followed by HKDF-SHA256 for session-key derivation.

The experiment is limited to key establishment and its associated computational and data-size costs. It does not, by itself, evaluate network-level latency, packet loss, throughput under concurrent load, resistance to implementation side channels, or end-to-end security against every class of attack.

AES-256-GCM remains the existing symmetric authenticated-encryption mechanism in the classical cryptographic pipeline. Its performance is not the primary subject of this experiment.

1.5 Comparison Methodology

The two mechanisms do not expose identical operations:

- **ML-KEM-768:** Generate a key pair, encapsulate to a recipient's public key, and decapsulate using the recipient's private key.
- **X25519 + HKDF-SHA256:** Generate ephemeral key pairs, compute a shared secret, and derive a session key using HKDF-SHA256.

Consequently, individual operation timings will be reported separately. Where an end-to-end comparison is presented, the measured operation sequence and included costs will be explicitly defined. KEM encapsulation and decapsulation will not be treated as direct semantic equivalents of X25519 shared-secret computation.

1.6 Experimental Integrity

All reported measurements will be obtained from executed experiments rather than estimated or assumed values. The notebook will distinguish correctness tests from performance measurements and will report the relevant sample counts, summary statistics, and experimental limitations.

The implementation will use an established cryptographic library rather than a custom implementation of ML-KEM. Cryptographic randomness will be provided by the library's supported secure random-generation mechanisms. Benchmark reproducibility will refer to reproducible procedures and recorded environments, not deterministic cryptographic keys or ciphertexts.

No claim of quantum advantage, universal performance superiority, or complete post-quantum security will be inferred solely from these benchmarks.

1.7 Expected Outputs

The completed experiment will produce:

- Functional validation results for ML-KEM-768.
- Correctness and failure-case test results.
- Operation-level latency measurements for both mechanisms.
- Cryptographic object-size comparisons.
- Statistical summaries of repeated benchmark runs.
- Machine-readable benchmark results and experiment metadata.
- A research-oriented interpretation of the findings, limitations, and integration implications for AEGES-Q.

End of Section 1

2. Environment, Dependencies, and Reproducibility

2.1 Experimental Environment

All experiments will be conducted within the Python environment configured for AEGES-Q. The Python interpreter, operating system, cryptographic library versions, and relevant package versions will be recorded to document the execution environment.

2.2 Dependency Management

The experiment will use a maintained cryptographic library that implements ML-KEM-768 in accordance with NIST FIPS 203. The selected implementation must expose the required key-generation, encapsulation, and decapsulation operations.

The existing AEGES-Q classical cryptography dependencies will be inspected before additional packages are installed. Existing package versions will not be changed without a demonstrated requirement.

2.3 Reproducibility Protocol

The following experimental details will be recorded:

- Python and operating-system versions.
- Cryptographic library and dependency versions.
- Benchmark sample counts and warm-up policy.
- Timing methodology and summary statistics.
- Algorithm parameters and object sizes.
- Any relevant implementation or environment limitations.

Cryptographic operations will use the library's secure randomness facilities. The benchmark will not replace secure random generation with deterministic randomness merely to obtain repeatable outputs.

Reproducibility will instead be supported through documented procedures, recorded environment metadata, repeated measurements, and machine-readable results.

```
import sys
import platform
import importlib.metadata as metadata
from pathlib import Path
import json

PROJECT_ROOT = Path.cwd().resolve().parents[1]

print("=" * 72)
print("AEGES-Q | PQC BENCHMARK ENVIRONMENT")
print("=" * 72)

environment = {
    "python_version": sys.version,
    "python_executable": sys.executable,
    "platform": platform.platform(),
    "machine": platform.machine(),
    "project_root": str(PROJECT_ROOT),
}

for key, value in environment.items():
    print(f"{key}: {value}")

print("\nRelevant installed packages:")
package_names = [
    "cryptography",
    "oqs",
    "liboqs-python",
    "pycryptodome",
    "numpy",
    "pandas",
    "scipy",
    "matplotlib",
]

installed_packages = {}

for package in package_names:
    try:
        version = metadata.version(package)
        installed_packages[package] = version
        print(f"{package}: {version}")
    except metadata.PackageNotFoundError:
        installed_packages[package] = None
        print(f"{package}: NOT INSTALLED")

print("\nProject root exists:", PROJECT_ROOT.exists())
```

```
print("Requirements file exists:", (PROJECT_ROOT /
"requirements.txt").exists())
```

```
================================================================================
AEGES-Q | PQC BENCHMARK ENVIRONMENT
================================================================================
python_version: 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11)
[MSC v.1944 64 bit (AMD64)]
python_executable: c:\Projects\Aeges-Q\.venv\Scripts\python.exe
platform: Windows-11-10.0.26200-SP0
machine: AMD64
project_root: C:\Projects\Aeges-Q

Relevant installed packages:
cryptography: 50.0.1
oqs: NOT INSTALLED
liboqs-python: NOT INSTALLED
pycryptodome: NOT INSTALLED
numpy: 2.5.2
pandas: 3.0.5
scipy: 1.18.1
matplotlib: 3.11.1

Project root exists: True
Requirements file exists: True
```

```
# Section 2.3 — ML-KEM-768 API Availability and Smoke Test

from cryptography.hazmat.primitives.asymmetric.mlkem import (
    MLKEM768PrivateKey,
)

# Generate the recipient's key pair.
private_key = MLKEM768PrivateKey.generate()
public_key = private_key.public_key()

# Encapsulate a shared secret using the recipient's public key.
sender_secret, ciphertext = public_key.encapsulate()

# Recover the shared secret using the recipient's private key.
recipient_secret = private_key.decapsulate(ciphertext)

# Validate the fundamental KEM correctness property.
assert sender_secret == recipient_secret, (
    "ML-KEM-768 round-trip verification failed."
)

assert len(sender_secret) == 32, (
```

```
    f"Unexpected shared-secret length: {len(sender_secret)} bytes"
)

print("ML-KEM-768 API import: PASS")
print("Key generation: PASS")
print("Encapsulation: PASS")
print("Decapsulation: PASS")
print("Shared-secret equality: PASS")
print(f"Shared-secret length: {len(sender_secret)} bytes")
print(f"Ciphertext length: {len(ciphertext)} bytes")
```

```
ML-KEM-768 API import: PASS
Key generation: PASS
Encapsulation: PASS
Decapsulation: PASS
Shared-secret equality: PASS
Shared-secret length: 32 bytes
Ciphertext length: 1088 bytes
```

2.4 Selected Implementation and Initial Validation

The experiment uses the ML-KEM-768 implementation provided by the Python `cryptography` package, version 50.0.1, through its `cryptography.hazmat.primitives.asymmetric.mlkem` API.

An initial smoke test successfully completed key generation, encapsulation, and decapsulation. The sender and recipient derived identical 32-byte shared secrets, and the encapsulation ciphertext measured 1,088 bytes, consistent with the standardized ML-KEM-768 parameter set.

These results confirm that the required API is available and that a basic encapsulation–decapsulation round trip succeeds in the current environment. More extensive correctness, failure-case, and performance testing will be performed in subsequent sections.

The experiment records environment metadata separately from benchmark results. Private keys, shared secrets, and other secret cryptographic material will not be written to the metadata file.

```
# Section 2.4 — Persist Environment Metadata

from datetime import datetime, timezone

PQC_ARTIFACT_DIR = PROJECT_ROOT / "artifacts" / "pqc"
PQC_ARTIFACT_DIR.mkdir(parents=True, exist_ok=True)

environment_metadata = {
    "project": "AEGES-Q",
    "experiment": "ML-KEM-768 benchmark",
    "experiment_stage": "environment_validation",
    "timestamp_utc": datetime.now(timezone.utc).isoformat(),
    "python_version": platform.python_version(),
```

```
    "python_executable": sys.executable,
    "platform": platform.platform(),
    "machine": platform.machine(),
    "cryptography_version": metadata.version("cryptography"),
    "mlkem_algorithm": "ML-KEM-768",
    "mlkem_api": (
        "cryptography.hazmat.primitives.asymmetric.mlkem"
    ),
    "smoke_test": {
        "key_generation": "PASS",
        "encapsulation": "PASS",
        "decapsulation": "PASS",
        "shared_secret_equality": "PASS",
        "shared_secret_bytes": len(sender_secret),
        "ciphertext_bytes": len(ciphertext),
    },
}

ENVIRONMENT_METADATA_PATH = (
    PQC_ARTIFACT_DIR / "pqc_environment_metadata.json"
)

with ENVIRONMENT_METADATA_PATH.open("w", encoding="utf-8") as file:
    json.dump(environment_metadata, file, indent=4)

print("Environment metadata saved:")
print(ENVIRONMENT_METADATA_PATH)
```

```
Environment metadata saved:
C:\Projects\Aeges-Q\artifacts\pqc\pqc_environment_metadata.json
```

3. Cryptographic Background and Comparison Methodology

3.1 ML-KEM-768: Post-Quantum Key Encapsulation

The Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) is standardized by the National Institute of Standards and Technology (NIST) in FIPS 203. Its security is based on the computational difficulty of the Module Learning With Errors problem. ML-KEM is designed to provide secure key establishment against adversaries that may possess quantum computers, subject to the assumptions and implementation requirements of the standard.

ML-KEM defines three parameter sets: ML-KEM-512, ML-KEM-768, and ML-KEM-1024. This experiment evaluates ML-KEM-768, which corresponds to NIST security category 3.

A key-encapsulation mechanism allows a sender to establish a shared secret with a recipient using the recipient's public encapsulation key. It consists of three principal operations:

1. **Key generation:** The recipient generates an encapsulation key pair consisting of a public key and a private decapsulation key.
2. **Encapsulation:** The sender uses the recipient's public key to generate a shared secret and an associated ciphertext.
3. **Decapsulation:** The recipient uses the private decapsulation key and ciphertext to recover the same shared secret.

The ciphertext can be transmitted over a public channel. The resulting shared secret can subsequently be used with symmetric cryptographic mechanisms, such as AES-256-GCM, for authenticated encryption.

3.1.1 Standardized ML-KEM-768 Object Sizes

According to NIST FIPS 203, the encoded sizes for ML-KEM-768 are:

Object	Standardized size
Encapsulation (public) key	1,184 bytes
Decapsulation key, expanded representation	2,400 bytes
Encapsulation ciphertext	1,088 bytes
Shared secret	32 bytes

Representation note: The private-key size depends on the representation being reported. The `cryptography` API's `private_bytes_raw()` method returns the 64-byte seed representation, not the expanded 2,400-byte decapsulation-key representation. These representations must not be reported as interchangeable.

The standardized sizes provide reference values against which the experimental implementation will be checked.

3.2 Classical Key Establishment: X25519 and HKDF-SHA256

The existing AEGES-Q classical cryptographic baseline uses X25519 for elliptic-curve Diffie–Hellman key agreement, followed by HKDF-SHA256 for key derivation.

X25519 allows two parties to derive a common shared secret by exchanging public keys derived from their respective private keys. Both parties perform a shared-secret computation using their own private key and the other party's public key.

The resulting shared secret is then passed through HKDF-SHA256, a key derivation function based on HMAC-SHA256. HKDF separates the process of deriving application keys from the raw key-agreement output and can incorporate context-specific information through its salt and information parameters.

The baseline workflow is:

1. Generate the required X25519 key pairs.
2. Exchange the public keys.
3. Compute the shared secret.
4. Derive the application session key using HKDF-SHA256.

The benchmark must document whether key generation, shared-secret computation, and HKDF execution are timed independently or together. An ephemeral key-establishment measurement must also specify which party's operations are included.

3.3 Fundamental Differences Between the Mechanisms

ML-KEM and X25519 + HKDF-SHA256 serve a related purpose but have different interfaces and security foundations.

Property	ML-KEM-768	X25519 + HKDF-SHA256
Cryptographic construction	Module-lattice-based KEM	Elliptic-curve key agreement plus KDF
Primary operation	Encapsulation and decapsulation	Shared-secret computation
Sender-side output	Ciphertext and shared secret	Shared secret after key agreement
Recipient-side operation	Decapsulation	Shared-secret computation
Key derivation	ML-KEM produces a 32-byte shared secret	HKDF derives the application key
Quantum resistance	Designed for post-quantum security	X25519 is vulnerable to sufficiently capable cryptographically relevant quantum attacks
Data exchanged	ML-KEM encapsulation ciphertext and public key material	X25519 public keys

The ML-KEM shared secret and X25519 shared-secret output are not identical in their cryptographic role or security assumptions. HKDF is an additional operation in the classical pipeline, while ML-KEM internally produces the shared secret through its standardized encapsulation mechanism.

3.4 Benchmarking Strategy

The experiment uses two complementary measurement levels.

3.4.1 Operation-Level Measurements

For ML-KEM-768, measure:

- Key-pair generation latency.
- Encapsulation latency.
- Decapsulation latency.

For the classical baseline, measure:

- X25519 key-pair generation latency.
- X25519 shared-secret computation latency.

- HKDF-SHA256 derivation latency.

Each operation will be measured independently where practical. The implementation will be initialized outside timed regions when initialization is not part of the operation under study.

3.4.2 Workflow-Level Measurements

Workflow measurements will capture the cost of a defined sequence of operations rather than treating unlike individual operations as equivalent.

For ML-KEM, the sender-side sequence includes encapsulation, while the recipient-side sequence includes decapsulation. Key generation will be reported separately and included in a workflow total only when explicitly defined.

For the classical baseline, a complete session-establishment measurement will specify the key-generation, public-key exchange assumptions, shared-secret computation, and HKDF operations included in the measured sequence.

Local computation time and network transmission time will be treated separately. A local benchmark does not directly measure network latency.

3.5 Experimental Controls

To make the comparison interpretable:

1. Both mechanisms will run on the same machine and Python environment.
2. Repeated measurements will be collected for each operation.
3. Warm-up iterations will be separated from recorded measurements.
4. High-resolution timing will be used for short cryptographic operations.
5. The benchmark will report sample counts, median latency, mean latency, standard deviation, and selected percentile statistics.
6. Public-key and ciphertext sizes will be measured independently of execution time.
7. The same cryptographic operation will not be timed together with printing, file I/O, or unrelated processing.
8. Randomized cryptographic operations will use the library's supported secure randomness facilities.
9. Benchmark results will include the implementation and dependency versions.
10. Results will be interpreted as measurements of the tested software and hardware configuration, not as universal algorithm rankings.

3.6 Interpretation and Limitations

The experiment evaluates the practical performance and data-size characteristics of two key-establishment approaches. It does not establish that either mechanism is universally faster, nor does a timing comparison alone establish security.

ML-KEM-768's standardized post-quantum security properties differ fundamentally from the classical security properties of X25519. This experiment will therefore report performance separately from security rationale.

The results will also be limited by the Python API, underlying cryptographic backend, operating system, hardware, system load, and measurement procedure. Any claim about AEGES-Q integration will be restricted to conclusions supported by the observed measurements.

End of Section 3

4. Implementation and API Validation

4.1 Implementation Overview

The experiment uses the ML-KEM-768 API provided by the `cryptography` package. The implementation exposes key generation through `MLKEM768PrivateKey.generate()`, public-key access through `public_key()`, encapsulation through `encapsulate()`, and decapsulation through `decapsulate()`.

The implementation is treated as a cryptographic dependency rather than reimplementing the underlying algorithm. This reduces the risk of introducing errors into security-critical mathematical operations.

4.2 Functional Interface

The benchmark will use the following logical operations:

- `generate_mlkem_keypair()`: Generate a recipient key pair.
- `encapsulate_mlkem(public_key)`: Generate a shared secret and ciphertext.
- `decapsulate_mlkem(private_key, ciphertext)`: Recover the shared secret.
- `validate_mlkem_round_trip()`: Confirm that encapsulation and decapsulation produce matching shared secrets.

The wrapper functions will remain thin so that the measured operations correspond closely to the underlying library calls. Input validation, logging, printing, and file I/O will be kept outside timed regions unless explicitly included in a separate experiment.

4.3 Correctness Requirements

The implementation must satisfy these basic requirements before performance measurements begin:

1. Key generation returns a usable private key and public key.
2. Encapsulation produces a ciphertext and a 32-byte shared secret.
3. Decapsulation recovers the same shared secret.
4. The ciphertext length matches the standardized ML-KEM-768 size of 1,088 bytes.
5. Public-key and private-key encodings are handled using documented library APIs.

A successful round trip is a functional check, not a complete cryptographic conformance or security assessment. Additional failure-case tests are specified in Section 5.

```python
# Section 4.2 — ML-KEM-768 Functional Interface

from cryptography.hazmat.primitives.asymmetric.mlkem import (
    MLKEM768PrivateKey,
)

MLKEM_SHARED_SECRET_BYTES = 32
MLKEM768_PUBLIC_KEY_BYTES = 1184
MLKEM768_CIPHERTEXT_BYTES = 1088


def generate_mlkem_keypair():
    """Generate an ML-KEM-768 private key and its public key."""
    private_key = MLKEM768PrivateKey.generate()
    public_key = private_key.public_key()
    return private_key, public_key


def encapsulate_mlkem(public_key):
    """Encapsulate to a recipient public key."""
    shared_secret, ciphertext = public_key.encapsulate()
    return shared_secret, ciphertext


def decapsulate_mlkem(private_key, ciphertext):
    """Decapsulate a ciphertext using the recipient private key."""
    return private_key.decapsulate(ciphertext)


def validate_mlkem_round_trip():
    """Validate one ML-KEM-768 encapsulation-decapsulation round
trip."""
    private_key, public_key = generate_mlkem_keypair()
    sender_secret, ciphertext = encapsulate_mlkem(public_key)
    recipient_secret = decapsulate_mlkem(private_key, ciphertext)

    assert sender_secret == recipient_secret
    assert len(sender_secret) == MLKEM_SHARED_SECRET_BYTES
    assert len(bytes(public_key.public_bytes_raw())) == (
        MLKEM768_PUBLIC_KEY_BYTES
    )
    assert len(ciphertext) == MLKEM768_CIPHERTEXT_BYTES

    return {
        "round_trip": "PASS",
        "public_key_bytes": len(bytes(public_key.public_bytes_raw())),
        "ciphertext_bytes": len(ciphertext),
        "shared_secret_bytes": len(sender_secret),
    }
```

```
mlkem_validation_result = validate_mlkem_round_trip()
print(mlkem_validation_result)
```

```
{'round_trip': 'PASS', 'public_key_bytes': 1184, 'ciphertext_bytes':
1088, 'shared_secret_bytes': 32}
```

4.3 Existing Classical Cryptography Baseline

The classical reference implementation is the existing AEGES-Q X25519 + HKDF-SHA256 key-establishment workflow.

Before implementing benchmark wrappers, the source implementation will be inspected to establish the exact operations performed, the cryptographic parameters, and the existing API boundaries.

The benchmark will preserve the original implementation and use thin wrappers around its existing functions wherever practical. Any differences between the existing implementation and the benchmark measurement procedure will be documented explicitly.

This inspection prevents accidental changes to the established baseline and ensures that subsequent comparisons reflect the project's actual implementation.

```
# Section 4.3 — Locate Existing Classical Crypto Source

source_dirs = [
    PROJECT_ROOT / "src",
    PROJECT_ROOT / "app",
]

crypto_candidates = []

for source_dir in source_dirs:
    if source_dir.exists():
        for path in source_dir.rglob("*.py"):
            searchable = str(path).lower()
            if any(term in searchable for term in ("crypto",
"security", "key")):
                crypto_candidates.append(path)

if crypto_candidates:
    for path in sorted(set(crypto_candidates)):
        print(path.relative_to(PROJECT_ROOT))
else:
    print("No candidate files found under src/ or app/.")
    print("We will inspect the existing crypto notebook next.")
```

```
app\backend\services\crypto_service.py
app\backend\services\key_exchange.py
```

```
# Section 4.4 — Inspect Existing Classical Crypto Source
```

```
crypto_source_files = [
    PROJECT_ROOT / "app" / "backend" / "services" /
"crypto_service.py",
    PROJECT_ROOT / "app" / "backend" / "services" / "key_exchange.py",
]

for source_path in crypto_source_files:
    print("\n" + "=" * 88)
    print(f"FILE: {source_path.relative_to(PROJECT_ROOT)}")
    print("=" * 88)

    if not source_path.exists():
        print("FILE NOT FOUND")
        continue

    source_text = source_path.read_text(encoding="utf-8")
    print(source_text)


========================================================================
==================
FILE: app\backend\services\crypto_service.py
========================================================================
==================
from __future__ import annotations

import os
from dataclasses import dataclass

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


AES_KEY_SIZE = 32           # 256 bits
AES_NONCE_SIZE = 12         # 96 bits, recommended for GCM
AES_TAG_SIZE = 16           # 128-bit authentication tag

DEFAULT_AAD = b"AEGES-Q|CLASSICAL|AES-256-GCM"


@dataclass(frozen=True)
class EncryptedPayload:
    """
    Authenticated AES-256-GCM ciphertext envelope.

    The nonce is stored separately from the ciphertext because
    the nonce is required for decryption but is not secret.
    """

    nonce: bytes
    ciphertext: bytes
```

```
class ClassicalCryptoService:
    """
    Production AES-256-GCM service for AEGES-Q.

    Responsibilities:
    - Generate AES-256 session keys
    - Encrypt application data
    - Decrypt authenticated ciphertext
    - Enforce correct nonce handling
    - Reject tampered ciphertext
    """

    def __init__(self, associated_data: bytes = DEFAULT_AAD):
        self.associated_data = associated_data

    @staticmethod
    def generate_key() -> bytes:
        """
        Generate a fresh 256-bit AES key.

        Key material is returned to the caller and is never logged
        or persisted by this service.
        """
        return AESGCM.generate_key(bit_length=256)

    @staticmethod
    def _validate_key(key: bytes) -> None:
        if not isinstance(key, bytes):
            raise TypeError("AES key must be bytes")

        if len(key) != AES_KEY_SIZE:
            raise ValueError("AES key must be exactly 32 bytes")

    def encrypt(
        self,
        plaintext: bytes,
        key: bytes,
    ) -> EncryptedPayload:
        """
        Encrypt plaintext using AES-256-GCM.

        A fresh 96-bit nonce is generated for every encryption.
        """

        if not isinstance(plaintext, bytes):
            raise TypeError("Plaintext must be bytes")

        self._validate_key(key)
```

```
        nonce = os.urandom(AES_NONCE_SIZE)

        aesgcm = AESGCM(key)

        ciphertext = aesgcm.encrypt(
            nonce,
            plaintext,
            self.associated_data,
        )

        return EncryptedPayload(
            nonce=nonce,
            ciphertext=ciphertext,
        )

    def decrypt(
        self,
        payload: EncryptedPayload,
        key: bytes,
    ) -> bytes:
        """
        Decrypt and authenticate an AES-256-GCM payload.

        Invalid or tampered ciphertext raises InvalidTag.
        """

        self._validate_key(key)

        if not isinstance(payload, EncryptedPayload):
            raise TypeError("payload must be EncryptedPayload")

        if len(payload.nonce) != AES_NONCE_SIZE:
            raise ValueError("Nonce must be exactly 12 bytes")

        aesgcm = AESGCM(key)

        try:
            return aesgcm.decrypt(
                payload.nonce,
                payload.ciphertext,
                self.associated_data,
            )
        except InvalidTag:
            raise InvalidTag(
                "AES-GCM authentication failed"
            ) from None


======================================================================
```

```
==================
FILE: app\backend\services\key_exchange.py
================================================================================
==================
from __future__ import annotations

from dataclasses import dataclass

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric.x25519 import (
    X25519PrivateKey,
    X25519PublicKey,
)
from cryptography.hazmat.primitives.kdf.hkdf import HKDF


X25519_KEY_SIZE = 32
SESSION_KEY_SIZE = 32

DEFAULT_HKDF_INFO = b"AEGES-Q|CLASSICAL|SESSION-KEY"


@dataclass
class EphemeralKeyPair:
    """
    Ephemeral X25519 key pair.

    The private key must remain internal and must never be exposed
    through an API response or log.
    """

    private_key: X25519PrivateKey
    public_key: X25519PublicKey


class X25519KeyExchange:
    """
    Classical ephemeral key-establishment service.

    X25519 establishes a shared secret.
    HKDF-SHA256 derives the application session key.
    """

    def __init__(
        self,
        hkdf_info: bytes = DEFAULT_HKDF_INFO,
    ):
        self.hkdf_info = hkdf_info

    @staticmethod
    def generate_key_pair() -> EphemeralKeyPair:
```

```
        """
        Generate an ephemeral X25519 key pair.
        """

        private_key = X25519PrivateKey.generate()
        public_key = private_key.public_key()

        return EphemeralKeyPair(
            private_key=private_key,
            public_key=public_key,
        )

    @staticmethod
    def serialize_public_key(
        public_key: X25519PublicKey,
    ) -> bytes:
        """
        Serialize an X25519 public key for transmission.
        """

        return public_key.public_bytes_raw()

    @staticmethod
    def deserialize_public_key(
        public_key_bytes: bytes,
    ) -> X25519PublicKey:
        """
        Deserialize a transmitted X25519 public key.
        """

        if len(public_key_bytes) != X25519_KEY_SIZE:
            raise ValueError(
                "X25519 public key must be exactly 32 bytes"
            )

        return X25519PublicKey.from_public_bytes(
            public_key_bytes
        )

    def derive_session_key(
        self,
        private_key: X25519PrivateKey,
        peer_public_key: X25519PublicKey,
    ) -> bytes:
        """
        Perform X25519 agreement and derive a 256-bit session key
        using HKDF-SHA256.
        """

        shared_secret = private_key.exchange(
```

```
            peer_public_key
        )

        session_key = HKDF(
            algorithm=hashes.SHA256(),
            length=SESSION_KEY_SIZE,
            salt=None,
            info=self.hkdf_info,
        ).derive(shared_secret)

        return session_key
```

4.5 Classical Baseline: Existing AEGES-Q Implementation

The classical baseline is sourced directly from `app/backend/services/key_exchange.py`. It uses the `X25519KeyExchange` service and its existing `EphemeralKeyPair` abstraction.

The implementation generates ephemeral X25519 key pairs, computes a shared secret through X25519, and derives a 32-byte session key through HKDF-SHA256. The configured HKDF parameters are preserved: `salt=None` and `info=b"AEGES-Q|CLASSICAL|SESSION-KEY"`.

The benchmark will measure key generation, X25519 agreement, and HKDF-SHA256 derivation separately. A combined measurement will also be defined for the complete local workflow. The source implementation will not be modified.

For correctness validation, two independently generated X25519 key pairs will be used to establish matching session keys. The ML-KEM and classical results will be compared as key-establishment outputs, while retaining the distinction between a KEM-generated shared secret and an HKDF-derived session key.

```
# Section 4.6 — Classical X25519 + HKDF-SHA256 Validation

import sys

BACKEND_DIR = PROJECT_ROOT / "app" / "backend"
if str(BACKEND_DIR) not in sys.path:
    sys.path.insert(0, str(BACKEND_DIR))

from services.key_exchange import X25519KeyExchange

classical_exchange = X25519KeyExchange()

# Simulate two parties with independent ephemeral key pairs.
alice = classical_exchange.generate_key_pair()
bob = classical_exchange.generate_key_pair()

alice_session_key = classical_exchange.derive_session_key(
    alice.private_key,
    bob.public_key,
```

```
)
bob_session_key = classical_exchange.derive_session_key(
    bob.private_key,
    alice.public_key,
)

assert alice_session_key == bob_session_key
assert len(alice_session_key) == 32

alice_public_bytes = classical_exchange.serialize_public_key(
    alice.public_key
)
bob_public_bytes = classical_exchange.serialize_public_key(
    bob.public_key
)

assert len(alice_public_bytes) == 32
assert len(bob_public_bytes) == 32

print("Existing X25519 + HKDF-SHA256 baseline: PASS")
print(f"Alice public key: {len(alice_public_bytes)} bytes")
print(f"Bob public key: {len(bob_public_bytes)} bytes")
print(f"Derived session key: {len(alice_session_key)} bytes")
print("Both parties derive identical session keys: PASS")
```

```
Existing X25519 + HKDF-SHA256 baseline: PASS
Alice public key: 32 bytes
Bob public key: 32 bytes
Derived session key: 32 bytes
Both parties derive identical session keys: PASS
```

5. Correctness, Interoperability, and Failure-Case Testing

5.1 Testing Objectives

Before collecting performance measurements, both cryptographic mechanisms must pass functional validation. These tests establish that the operations produce the expected outputs and that the two-party key-establishment workflows behave correctly.

The tests cover:

1. ML-KEM-768 encapsulation and decapsulation consistency.
2. X25519 + HKDF-SHA256 two-party session-key agreement.
3. Standardized key, ciphertext, and shared-secret output sizes.
4. Repeated independent ML-KEM encapsulation-decapsulation trials.
5. X25519 public-key serialization and deserialization.
6. Rejection of malformed ML-KEM ciphertexts by the underlying library, where supported by its API contract.
7. Rejection of invalid X25519 public-key encodings by the existing project service.

5.2 Test Interpretation

Successful tests establish functional properties for the tested software and environment. They do not constitute a formal validation of the cryptographic implementation, a proof of security, or a complete assessment of side-channel resistance.

Failure behaviour must be interpreted according to each API's documented contract. In particular, an ML-KEM decapsulation operation may not reject every modified ciphertext by raising an exception; the test must not assume that ciphertext tampering always produces a detectable error.

The tests will distinguish required correctness properties from implementation-specific error handling. Any unexpected result will be recorded and investigated before benchmarking proceeds.

```
# Section 5.2 — Repeated ML-KEM-768 Correctness Tests

MLKEM_CORRECTNESS_TRIALS = 100

mlkem_test_results = []

for trial in range(MLKEM_CORRECTNESS_TRIALS):
    private_key, public_key = generate_mlkem_keypair()
    sender_secret, ciphertext = encapsulate_mlkem(public_key)
    recipient_secret = decapsulate_mlkem(private_key, ciphertext)

    assert sender_secret == recipient_secret, (
        f"Shared-secret mismatch in trial {trial}"
    )
    assert len(sender_secret) == MLKEM_SHARED_SECRET_BYTES
    assert len(bytes(public_key.public_bytes_raw())) == (
        MLKEM768_PUBLIC_KEY_BYTES
    )
    assert len(ciphertext) == MLKEM768_CIPHERTEXT_BYTES

    mlkem_test_results.append({
        "trial": trial + 1,
        "round_trip": True,
        "public_key_bytes": len(bytes(public_key.public_bytes_raw())),
        "ciphertext_bytes": len(ciphertext),
        "shared_secret_bytes": len(sender_secret),
    })

print(f"ML-KEM-768 correctness trials: {len(mlkem_test_results)}")
print("Successful round trips:", sum(r["round_trip"] for r in
mlkem_test_results))
print("All size and shared-secret checks passed.")
```

```
ML-KEM-768 correctness trials: 100
Successful round trips: 100
All size and shared-secret checks passed.
```

```python
# Section 5.3 — X25519 Serialization and Invalid-Input Tests

from cryptography.hazmat.primitives.asymmetric.x25519 import (
    X25519PublicKey,
)

# Test public-key serialization and deserialization.
original_pair = classical_exchange.generate_key_pair()
serialized_public_key = classical_exchange.serialize_public_key(
    original_pair.public_key
)
restored_public_key = classical_exchange.deserialize_public_key(
    serialized_public_key
)

assert restored_public_key.public_bytes_raw() == serialized_public_key
print("X25519 public-key serialization round trip: PASS")

# Test rejection of an incorrectly sized public-key encoding.
try:
    classical_exchange.deserialize_public_key(b"\x00" * 31)
except ValueError:
    print("Malformed X25519 public-key length rejection: PASS")
else:
    raise AssertionError(
        "Expected ValueError for a 31-byte X25519 public key."
    )

# Test that independently generated parties derive the same session
key.
party_a = classical_exchange.generate_key_pair()
party_b = classical_exchange.generate_key_pair()

key_a = classical_exchange.derive_session_key(
    party_a.private_key, party_b.public_key
)
key_b = classical_exchange.derive_session_key(
    party_b.private_key, party_a.public_key
)

assert key_a == key_b
assert len(key_a) == 32
print("X25519 + HKDF two-party agreement: PASS")
print("Classical derived session-key length: 32 bytes")
```

```
X25519 public-key serialization round trip: PASS
Malformed X25519 public-key length rejection: PASS
X25519 + HKDF two-party agreement: PASS
Classical derived session-key length: 32 bytes
```

```python
# Section 5.4 — ML-KEM-768 Malformed-Ciphertext Behaviour

private_key, public_key = generate_mlkem_keypair()
sender_secret, valid_ciphertext = encapsulate_mlkem(public_key)

# A ciphertext with an invalid length should be rejected.
try:
    private_key.decapsulate(valid_ciphertext[:-1])
except (ValueError, TypeError) as exc:
    malformed_length_result = {
        "status": "REJECTED",
        "exception": type(exc).__name__,
    }
else:
    raise AssertionError(
        "The API accepted a ciphertext with an invalid length."
    )

print("Invalid-length ciphertext:", malformed_length_result)

# A same-length modified ciphertext is a separate test.
modified_ciphertext = bytearray(valid_ciphertext)
modified_ciphertext[0] ^= 0x01
modified_ciphertext = bytes(modified_ciphertext)

try:
    modified_secret = private_key.decapsulate(modified_ciphertext)
    modified_ciphertext_result = {
        "status": "RETURNED_SECRET",
        "secret_length": len(modified_secret),
        "matches_original": modified_secret == sender_secret,
    }
except Exception as exc:
    modified_ciphertext_result = {
        "status": "RAISED_EXCEPTION",
        "exception": type(exc).__name__,
    }

print("Modified same-length ciphertext:", modified_ciphertext_result)
print(
    "Interpretation: record the observed API behaviour; "
    "do not infer authentication from this test alone."
)
```

```
Invalid-length ciphertext: {'status': 'REJECTED', 'exception':
'ValueError'}
Modified same-length ciphertext: {'status': 'RETURNED_SECRET',
'secret_length': 32, 'matches_original': False}
Interpretation: record the observed API behaviour; do not infer
authentication from this test alone.
```

5.5 Correctness Test Results and Interpretation

The correctness tests produced the following observations:

Test	Observed result
ML-KEM-768 round trips	100/100 successful
ML-KEM public-key size	1,184 bytes
ML-KEM ciphertext size	1,088 bytes
ML-KEM shared-secret size	32 bytes
X25519 + HKDF two-party agreement	Successful
X25519 public-key serialization	Successful
Invalid-length ML-KEM ciphertext	Rejected with `ValueError`
Modified same-length ML-KEM ciphertext	Returned a 32-byte secret different from the original

The successful ML-KEM round trips establish the expected encapsulation-decapsulation consistency for the tested trials. The classical tests establish successful two-party session-key derivation and public-key serialization using the existing AEGES-Q implementation.

The malformed-ciphertext experiment demonstrates that the installed ML-KEM API rejects the tested invalid-length input. For the tested same-length modification, decapsulation returned a secret different from the original shared secret rather than raising an exception.

This behaviour must be interpreted in accordance with the ML-KEM specification and the library's documented API contract. ML-KEM is a key-encapsulation mechanism, not an authenticated-encryption scheme. The experiment does not establish that every possible malformed ciphertext will be handled identically, nor does it assess the implementation's resistance to side-channel attacks.

These tests are functional checks, not a formal cryptographic validation or proof of security. The benchmark will proceed using the verified APIs, with correctness testing kept separate from performance measurement.

End of Section 5

6. Benchmark Methodology and Experimental Controls

6.1 Objective

This section defines the timing methodology, repetition strategy, statistical summaries, and experimental controls used to evaluate ML-KEM-768 and the existing X25519 + HKDF-SHA256 implementation.

The protocol is established before the main benchmark runs to reduce the risk of selecting measurement procedures based on observed results.

6.2 Timing Method

Execution time will be measured using Python's `time.perf_counter_ns()`, a high-resolution monotonic performance counter. Elapsed time will be calculated from the difference between the counter readings immediately before and after the operation.

The measured duration will be converted to milliseconds for reporting.

Input preparation, output formatting, file I/O, and unrelated processing will remain outside the timed region. Each measurement will target a specific cryptographic operation or a clearly defined workflow.

6.3 Warm-Up and Repeated Measurements

Each operation will undergo a warm-up phase before recorded measurements. Warm-up observations will not be included in the reported sample statistics.

Recorded measurements will use repeated independent trials. Each trial will use valid cryptographic inputs generated through the library's supported interfaces. For operations requiring a key pair or ciphertext, the benchmark will prepare the necessary inputs outside the timed region unless the operation being measured explicitly includes their generation.

The number of warm-up and recorded iterations will be fixed before the primary benchmark run and saved with the experiment metadata.

6.4 Operations to Be Measured

The primary operation-level measurements are:

ML-KEM-768

- Key-pair generation.
- Encapsulation using a pre-generated recipient public key.
- Decapsulation using a pre-generated recipient private key and valid ciphertext.

X25519 + HKDF-SHA256

- Ephemeral key-pair generation.
- X25519 shared-secret computation.
- HKDF-SHA256 derivation from a precomputed shared secret.

The combined classical session-key derivation operation will also be measured using the existing `X25519KeyExchange.derive_session_key()` implementation. This combined measurement includes both X25519 agreement and HKDF-SHA256 derivation.

6.5 Workflow-Level Measurements

Workflow measurements will be reported separately from individual operation timings.

The ML-KEM sender-side operation consists of encapsulation, while the recipient-side operation consists of decapsulation. Key generation will be reported separately unless explicitly included in a defined workflow.

The classical workflow uses ephemeral X25519 key pairs and HKDF-SHA256. The benchmark will state which key-generation, agreement, and derivation operations are included in each aggregate measurement.

The local computational measurements do not include network transmission time, protocol negotiation, application-level retries, or the cost of establishing a real remote connection.

6.6 Statistical Analysis

For each operation, the benchmark will report:

- Number of recorded measurements.
- Mean latency.
- Median latency.
- Standard deviation.
- Minimum and maximum observed latency.
- 95th percentile latency.

The median will serve as the primary descriptive measure of typical observed latency because short-duration measurements can be affected by occasional system interruptions. The mean and standard deviation will be retained to describe the distribution of observations.

Percentile estimates will be calculated from the recorded sample distribution. They will be interpreted descriptively rather than as formal guarantees about future execution times.

6.7 Experimental Controls and Limitations

All primary measurements will use the same Python environment and physical machine. The experiment will record the Python version, operating system, cryptographic package version, iteration counts, and benchmark configuration.

The benchmark will avoid printing or writing files during timed operations. Cryptographic randomness will remain enabled through the library's supported interfaces.

Because the measurements are collected on a general-purpose operating system, background activity and system scheduling can influence latency. The results characterize the tested implementation and environment rather than providing universal hardware-independent rankings.

6.8 Reproducibility and Result Preservation

The benchmark configuration, environment metadata, and raw timing observations will be preserved as artifacts under `artifacts/pqc/`. Summary tables will be derived from the raw measurements rather than manually entered.

The experiment will distinguish raw observations from derived statistics and will retain the operation definitions used to generate each result. This allows later analysis without rerunning every cryptographic operation.

```python
# Section 6.2 — Predefined Benchmark Configuration

import time
import statistics
import numpy as np

BENCHMARK_CONFIG = {
    "experiment_id": "aeges_q_mlkem768_vs_x25519_hkdf_v1",
    "timing_function": "time.perf_counter_ns",
    "warmup_iterations": 20,
    "recorded_iterations": 500,
    "latency_unit": "milliseconds",
    "summary_statistics": [
        "count",
        "mean",
        "median",
        "standard_deviation",
        "minimum",
        "maximum",
        "p95",
    ],
    "operations": [
        "mlkem768_key_generation",
        "mlkem768_encapsulation",
        "mlkem768_decapsulation",
        "x25519_key_generation",
        "x25519_shared_secret",
        "hkdf_sha256_derivation",
        "x25519_hkdf_combined_derivation",
    ],
}

BENCHMARK_CONFIG_PATH = (
    PQC_ARTIFACT_DIR / "pqc_benchmark_config.json"
)

with BENCHMARK_CONFIG_PATH.open("w", encoding="utf-8") as file:
    json.dump(BENCHMARK_CONFIG, file, indent=4)

print("Benchmark configuration saved:")
print(BENCHMARK_CONFIG_PATH)
print(json.dumps(BENCHMARK_CONFIG, indent=2))
```

```
Benchmark configuration saved:
C:\Projects\Aeges-Q\artifacts\pqc\pqc_benchmark_config.json
{
  "experiment_id": "aeges_q_mlkem768_vs_x25519_hkdf_v1",
  "timing_function": "time.perf_counter_ns",
  "warmup_iterations": 20,
  "recorded_iterations": 500,
```

```
  "latency_unit": "milliseconds",
  "summary_statistics": [
    "count",
    "mean",
    "median",
    "standard_deviation",
    "minimum",
    "maximum",
    "p95"
  ],
  "operations": [
    "mlkem768_key_generation",
    "mlkem768_encapsulation",
    "mlkem768_decapsulation",
    "x25519_key_generation",
    "x25519_shared_secret",
    "hkdf_sha256_derivation",
    "x25519_hkdf_combined_derivation"
  ]
}

# Section 6.3 — Reusable Timing and Statistical Summary Functions

def time_operation(operation):
    """Measure one operation's wall-clock duration in milliseconds."""
    start_ns = time.perf_counter_ns()
    result = operation()
    end_ns = time.perf_counter_ns()
    return (end_ns - start_ns) / 1_000_000, result


def benchmark_operation(operation, warmups=20, iterations=500):
    """Warm up an operation, then collect recorded latency samples."""
    for _ in range(warmups):
        operation()

    latencies_ms = []

    for _ in range(iterations):
        elapsed_ms, _ = time_operation(operation)
        latencies_ms.append(elapsed_ms)

    return latencies_ms


def summarize_latencies(latencies_ms):
    """Calculate descriptive statistics for recorded latency
samples."""
    values = np.asarray(latencies_ms, dtype=float)
```

```
    if values.size == 0:
        raise ValueError("At least one latency measurement is
required.")

    return {
        "count": int(values.size),
        "mean_ms": float(np.mean(values)),
        "median_ms": float(np.median(values)),
        "std_ms": float(np.std(values, ddof=1)) if values.size > 1
else 0.0,
        "min_ms": float(np.min(values)),
        "max_ms": float(np.max(values)),
        "p95_ms": float(np.percentile(values, 95)),
    }


# Validate the harness using a trivial operation, not a crypto
benchmark.
smoke_latencies = benchmark_operation(
    lambda: sum(range(100)),
    warmups=3,
    iterations=10,
)

smoke_summary = summarize_latencies(smoke_latencies)

assert smoke_summary["count"] == 10
assert smoke_summary["median_ms"] >= 0

print("Timing harness validation: PASS")
print("Smoke-test sample count:", smoke_summary["count"])
print("Statistics produced:", list(smoke_summary.keys()))
```

```
Timing harness validation: PASS
Smoke-test sample count: 10
Statistics produced: ['count', 'mean_ms', 'median_ms', 'std_ms',
'min_ms', 'max_ms', 'p95_ms']
```

6.4 Operation Definitions and Timing Boundaries

Each benchmark operation is defined by its timed boundary. Inputs and cryptographic objects required by an operation will be prepared outside the timed region unless their generation is the operation being measured.

ML-KEM-768

- **Key generation:** Time `MLKEM768PrivateKey.generate()`, including creation of the private key and its associated public key.
- **Encapsulation:** Time `public_key.encapsulate()` using a pre-generated recipient public key.

- **Decapsulation:** Time `private_key.decapsulate(ciphertext)` using a pre-generated private key and valid ciphertext.

X25519 + HKDF-SHA256

- **Key generation:** Time `X25519KeyExchange.generate_key_pair()`.
- **Shared-secret computation:** Time the X25519 exchange operation using a pre-generated private key and peer public key.
- **HKDF-SHA256 derivation:** Time HKDF derivation using a precomputed shared secret and the existing baseline parameters.
- **Combined derivation:** Time `X25519KeyExchange.derive_session_key()`, which performs X25519 agreement followed by HKDF-SHA256 derivation.

For operation-level measurements, key pairs and ciphertexts will be generated outside the timed region when they are not part of the operation being measured. Each decapsulation measurement will use a valid ciphertext generated for the corresponding private key.

The combined X25519 measurement represents one party's shared-secret computation and session-key derivation with an already available peer public key. It does not include peer key generation or network transmission.

These boundaries will be retained in the result metadata so that individual timings and workflow-level totals are not conflated.

```
# Section 6.5 — Define Benchmark Operations

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

def mlkem_key_generation_operation():
    return MLKEM768PrivateKey.generate()


mlkem_bench_private, mlkem_bench_public = generate_mlkem_keypair()
mlkem_bench_secret, mlkem_bench_ciphertext = encapsulate_mlkem(
    mlkem_bench_public
)

def mlkem_encapsulation_operation():
    return mlkem_bench_public.encapsulate()


def mlkem_decapsulation_operation():
    return mlkem_bench_private.decapsulate(mlkem_bench_ciphertext)


def x25519_key_generation_operation():
    return classical_exchange.generate_key_pair()


x25519_bench_alice = classical_exchange.generate_key_pair()
```

```
x25519_bench_bob = classical_exchange.generate_key_pair()

x25519_bench_shared_secret = x25519_bench_alice.private_key.exchange(
    x25519_bench_bob.public_key
)


def x25519_shared_secret_operation():
    return x25519_bench_alice.private_key.exchange(
        x25519_bench_bob.public_key
    )


def hkdf_sha256_operation():
    return HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=classical_exchange.hkdf_info,
    ).derive(x25519_bench_shared_secret)


def x25519_hkdf_combined_operation():
    return classical_exchange.derive_session_key(
        x25519_bench_alice.private_key,
        x25519_bench_bob.public_key,
    )


# Sanity-check outputs before timing.
assert len(mlkem_decapsulation_operation()) == 32
assert len(x25519_shared_secret_operation()) == 32
assert len(hkdf_sha256_operation()) == 32
assert len(x25519_hkdf_combined_operation()) == 32

print("Operation definitions: PASS")
print("ML-KEM decapsulation input: valid ciphertext")
print("Classical HKDF info: ", classical_exchange.hkdf_info)
print("All tested derivation outputs: 32 bytes")
```

```
Operation definitions: PASS
ML-KEM decapsulation input: valid ciphertext
Classical HKDF info:  b'AEGES-Q|CLASSICAL|SESSION-KEY'
All tested derivation outputs: 32 bytes
```

6.6 Final Benchmark Execution Protocol

The primary benchmark will collect 500 recorded measurements per operation after 20 warm-up iterations. Timing observations will be recorded in milliseconds using `time.perf_counter_ns()`.

For key generation, each timed invocation generates new key material. For encapsulation, the recipient public key is generated before timing, and each invocation performs a new encapsulation. For decapsulation, valid ciphertexts will be prepared for the corresponding private key outside the timed region, and each timed invocation will decapsulate a valid ciphertext.

The classical key-generation operation will generate a new ephemeral X25519 key pair per invocation. Shared-secret computation and HKDF derivation will be measured separately using prepared inputs. The combined X25519 + HKDF measurement will use the existing AEGES-Q service method.

Raw latency samples will be preserved in a machine-readable file. Summary statistics will be derived from those samples. The benchmark configuration and environment metadata will accompany the results.

The measurements characterize local operation latency for the tested software and hardware environment. They exclude network transmission, remote processing, and end-to-end application overhead unless explicitly stated otherwise.

7. Performance Benchmarking

7.1 Benchmark Execution

The benchmark evaluates the seven predefined operations using the configuration established in Section 6: 20 warm-up iterations and 500 recorded measurements per operation.

For each operation, the benchmark records the elapsed time of the cryptographic operation itself. Inputs required by the operation are prepared outside the timed region when their preparation is not part of the operation under study.

ML-KEM-768 encapsulation uses a pre-generated recipient public key. Decapsulation uses a valid ciphertext and the corresponding private key. The X25519 and HKDF-SHA256 operations use the existing AEGES-Q implementation and its configured derivation parameters.

Raw measurements will be saved before statistical summaries are calculated. This preserves the original observations and allows the results to be independently reanalysed.

```
# Section 7.2 — Execute the Cryptographic Benchmarks

import pandas as pd
from datetime import datetime, timezone

WARMUPS = BENCHMARK_CONFIG["warmup_iterations"]
ITERATIONS = BENCHMARK_CONFIG["recorded_iterations"]

raw_benchmark_results = {}
benchmark_summary_rows = []

def run_and_record(operation_name, operation):
    print(f"Running {operation_name} ...")
```

```
    samples = benchmark_operation(
        operation,
        warmups=WARMUPS,
        iterations=ITERATIONS,
    )
    raw_benchmark_results[operation_name] = samples

    summary = summarize_latencies(samples)
    benchmark_summary_rows.append({
        "operation": operation_name,
        **summary,
    })
    print(
        f"  completed: {summary['count']} samples | "
        f"median: {summary['median_ms']:.6f} ms"
    )


# 1. ML-KEM-768 key generation
run_and_record(
    "mlkem768_key_generation",
    mlkem_key_generation_operation,
)

# 2. ML-KEM-768 encapsulation using a fixed recipient public key
run_and_record(
    "mlkem768_encapsulation",
    mlkem_encapsulation_operation,
)

# 3. ML-KEM-768 decapsulation.
# Prepare a fresh key/ciphertext pair per recorded invocation.
decapsulation_cases = []
for _ in range(WARMUPS + ITERATIONS):
    private_key, public_key = generate_mlkem_keypair()
    shared_secret, ciphertext = encapsulate_mlkem(public_key)
    decapsulation_cases.append((private_key, ciphertext,
shared_secret))

decapsulation_index = {"value": 0}

def mlkem_decapsulation_prepared_operation():
    private_key, ciphertext, expected_secret = decapsulation_cases[
        decapsulation_index["value"]
    ]
    decapsulation_index["value"] += 1
    recovered_secret = private_key.decapsulate(ciphertext)
    assert recovered_secret == expected_secret
    return recovered_secret
```

```
run_and_record(
    "mlkem768_decapsulation",
    mlkem_decapsulation_prepared_operation,
)

# 4. X25519 key generation
run_and_record(
    "x25519_key_generation",
    x25519_key_generation_operation,
)

# 5. X25519 shared-secret computation
run_and_record(
    "x25519_shared_secret",
    x25519_shared_secret_operation,
)

# 6. HKDF-SHA256 derivation from a precomputed X25519 shared secret
run_and_record(
    "hkdf_sha256_derivation",
    hkdf_sha256_operation,
)

# 7. Combined X25519 agreement + HKDF-SHA256 derivation
run_and_record(
    "x25519_hkdf_combined_derivation",
    x25519_hkdf_combined_operation,
)

# Preserve raw observations and derived statistics.
benchmark_timestamp = datetime.now(timezone.utc).isoformat()

raw_rows = []
for operation_name, samples in raw_benchmark_results.items():
    for sample_number, latency_ms in enumerate(samples, start=1):
        raw_rows.append({
            "experiment_id": BENCHMARK_CONFIG["experiment_id"],
            "timestamp_utc": benchmark_timestamp,
            "operation": operation_name,
            "sample": sample_number,
            "latency_ms": latency_ms,
        })

raw_results_df = pd.DataFrame(raw_rows)
benchmark_summary_df = pd.DataFrame(benchmark_summary_rows)

RAW_RESULTS_PATH = PQC_ARTIFACT_DIR / "pqc_raw_benchmark_results.csv"
SUMMARY_RESULTS_PATH = PQC_ARTIFACT_DIR / "pqc_benchmark_summary.csv"

raw_results_df.to_csv(RAW_RESULTS_PATH, index=False)
```

```python
benchmark_summary_df.to_csv(SUMMARY_RESULTS_PATH, index=False)

print("\n" + "=" * 72)
print("BENCHMARK COMPLETE")
print("=" * 72)
print(f"Raw observations: {len(raw_results_df)}")
print(f"Raw results saved: {RAW_RESULTS_PATH}")
print(f"Summary saved: {SUMMARY_RESULTS_PATH}")

display(
    benchmark_summary_df[
        [
            "operation",
            "count",
            "mean_ms",
            "median_ms",
            "std_ms",
            "min_ms",
            "max_ms",
            "p95_ms",
        ]
    ].round(6)
)
```

```
Running mlkem768_key_generation ...
  completed: 500 samples | median: 0.378250 ms
Running mlkem768_encapsulation ...
  completed: 500 samples | median: 0.095900 ms
Running mlkem768_decapsulation ...
  completed: 500 samples | median: 0.143600 ms
Running x25519_key_generation ...
  completed: 500 samples | median: 0.089200 ms
Running x25519_shared_secret ...
  completed: 500 samples | median: 0.076800 ms
Running hkdf_sha256_derivation ...
  completed: 500 samples | median: 0.011100 ms
Running x25519_hkdf_combined_derivation ...
  completed: 500 samples | median: 0.087900 ms

========================================================================
==
BENCHMARK COMPLETE
========================================================================
==
Raw observations: 3500
Raw results saved: C:\Projects\Aeges-Q\artifacts\pqc\
pqc_raw_benchmark_results.csv
Summary saved: C:\Projects\Aeges-Q\artifacts\pqc\
pqc_benchmark_summary.csv
```

```
                          operation  count   mean_ms  median_ms
std_ms  \
0          mlkem768_key_generation    500  0.396462    0.37825
0.056953
1           mlkem768_encapsulation    500  0.118025    0.09590
0.067948
2           mlkem768_decapsulation    500  0.152672    0.14360
0.034206
3            x25519_key_generation    500  0.099183    0.08920
0.056764
4             x25519_shared_secret    500  0.079487    0.07680
0.011661
5           hkdf_sha256_derivation    500  0.011212    0.01110
0.001452
6  x25519_hkdf_combined_derivation    500  0.097671    0.08790
0.043264

   min_ms  max_ms    p95_ms
0  0.3619  0.8744  0.499795
1  0.0932  0.9421  0.231250
2  0.1415  0.4898  0.175275
3  0.0865  1.2214  0.123055
4  0.0757  0.1982  0.091700
5  0.0107  0.0359  0.011500
6  0.0868  0.6247  0.145315
```

7.3 Initial Performance Observations

The first benchmark run collected 500 recorded latency samples for each of seven operations, producing 3,500 raw observations.

The observed median ML-KEM-768 key-generation latency was 0.37825 ms, compared with 0.08920 ms for X25519 key generation. ML-KEM-768 encapsulation had a median latency of 0.09590 ms, and decapsulation had a median latency of 0.14360 ms.

For the classical implementation, X25519 shared-secret computation had a median latency of 0.07680 ms, while HKDF-SHA256 derivation had a median latency of 0.01110 ms. The existing combined X25519 + HKDF-SHA256 method had a median latency of 0.08790 ms.

These measurements indicate that the relative computational costs differ by operation. In particular, ML-KEM-768 key generation took longer than X25519 key generation in this run. Encapsulation and the combined classical derivation had median latencies of the same broad order of magnitude, although the measured values were not identical.

The results characterize the tested software and hardware environment. They do not establish universal performance rankings, and the variation observed across samples warrants examination alongside medians, means, standard deviations, and percentiles. Further interpretation will consider object sizes and the precise boundaries of each operation.

8. Cryptographic Object Sizes and Data Overhead

8.1 Objective

This section compares the sizes of the cryptographic objects produced or exchanged by ML-KEM-768 and the existing X25519 + HKDF-SHA256 implementation.

Object sizes will be obtained from the actual library encodings where possible and checked against standardized reference values. The comparison distinguishes public keys, private-key representations, ciphertexts, shared secrets, and derived session keys.

8.2 Interpretation of Size Measurements

ML-KEM-768 produces an encapsulation ciphertext and a 32-byte shared secret. Its public encapsulation key has a standardized encoded size of 1,184 bytes. The private key has multiple possible representations, so the representation and serialization method must be recorded explicitly.

X25519 public keys and raw private-key encodings are each 32 bytes. X25519 does not produce a KEM ciphertext; the parties exchange public keys to establish a shared secret. HKDF-SHA256 then derives the 32-byte application session key.

The size comparison therefore reports the actual encoded objects rather than treating an X25519 public key as semantically equivalent to an ML-KEM ciphertext. Protocol-level communication overhead will depend on which objects are transmitted, whether keys are reused, and the surrounding protocol.

```
# Section 8.2 — Cryptographic Object Size Comparison

from cryptography.hazmat.primitives import serialization

# ML-KEM-768 objects
mlkem_private_key, mlkem_public_key = generate_mlkem_keypair()
mlkem_shared_secret, mlkem_ciphertext = encapsulate_mlkem(
    mlkem_public_key
)

mlkem_public_key_bytes = mlkem_public_key.public_bytes_raw()
mlkem_private_seed_bytes = mlkem_private_key.private_bytes_raw()

# X25519 objects
x25519_pair = classical_exchange.generate_key_pair()
x25519_public_key_bytes = classical_exchange.serialize_public_key(
    x25519_pair.public_key
)
x25519_private_key_bytes = x25519_pair.private_key.private_bytes_raw()

# Classical shared secret and derived session key
x25519_peer = classical_exchange.generate_key_pair()
```

```
x25519_shared_secret_bytes = x25519_pair.private_key.exchange(
    x25519_peer.public_key
)
x25519_session_key_bytes = classical_exchange.derive_session_key(
    x25519_pair.private_key,
    x25519_peer.public_key,
)

object_sizes = [
    {
        "mechanism": "ML-KEM-768",
        "object": "Encapsulation public key",
        "representation": "Standard raw encoding",
        "size_bytes": len(mlkem_public_key_bytes),
    },
    {
        "mechanism": "ML-KEM-768",
        "object": "Private key",
        "representation": "cryptography private_bytes_raw() seed
representation",
        "size_bytes": len(mlkem_private_seed_bytes),
    },
    {
        "mechanism": "ML-KEM-768",
        "object": "Encapsulation ciphertext",
        "representation": "Standard ciphertext encoding",
        "size_bytes": len(mlkem_ciphertext),
    },
    {
        "mechanism": "ML-KEM-768",
        "object": "Shared secret",
        "representation": "Derived by encapsulation",
        "size_bytes": len(mlkem_shared_secret),
    },
    {
        "mechanism": "X25519 + HKDF-SHA256",
        "object": "Public key",
        "representation": "Raw X25519 public-key encoding",
        "size_bytes": len(x25519_public_key_bytes),
    },
    {
        "mechanism": "X25519 + HKDF-SHA256",
        "object": "Private key",
        "representation": "Raw X25519 private-key encoding",
        "size_bytes": len(x25519_private_key_bytes),
    },
    {
        "mechanism": "X25519 + HKDF-SHA256",
        "object": "Shared secret",
```

```
        "representation": "Raw X25519 exchange output",
        "size_bytes": len(x25519_shared_secret_bytes),
    },
    {
        "mechanism": "X25519 + HKDF-SHA256",
        "object": "Session key",
        "representation": "HKDF-SHA256 output",
        "size_bytes": len(x25519_session_key_bytes),
    },
]

object_sizes_df = pd.DataFrame(object_sizes)

OBJECT_SIZES_PATH = PQC_ARTIFACT_DIR /
"pqc_cryptographic_object_sizes.csv"
object_sizes_df.to_csv(OBJECT_SIZES_PATH, index=False)

display(object_sizes_df)
print(f"\nObject-size results saved to: {OBJECT_SIZES_PATH}")
```

```
              mechanism                    object  \
0            ML-KEM-768  Encapsulation public key
1            ML-KEM-768               Private key
2            ML-KEM-768  Encapsulation ciphertext
3            ML-KEM-768             Shared secret
4  X25519 + HKDF-SHA256                Public key
5  X25519 + HKDF-SHA256               Private key
6  X25519 + HKDF-SHA256             Shared secret
7  X25519 + HKDF-SHA256               Session key

                                    representation  size_bytes
0                              Standard raw encoding        1184
1  cryptography private_bytes_raw() seed represen...          64
2                        Standard ciphertext encoding        1088
3                            Derived by encapsulation          32
4                     Raw X25519 public-key encoding          32
5                    Raw X25519 private-key encoding          32
6                         Raw X25519 exchange output          32
7                                 HKDF-SHA256 output          32


Object-size results saved to: C:\Projects\Aeges-Q\artifacts\pqc\
pqc_cryptographic_object_sizes.csv
```

8.3 Communication Overhead Analysis

The encoded object sizes highlight a key distinction between the two mechanisms. ML-KEM-768 uses a larger encapsulation public key and produces a 1,088-byte ciphertext, whereas an X25519 public key is 32 bytes.

For an exchange in which the recipient's public key has already been provisioned, the principal key-establishment object transmitted by the ML-KEM sender is the encapsulation ciphertext. In the classical X25519 workflow, the parties exchange public keys to establish a shared secret.

The resulting communication overhead therefore depends on the protocol's key-distribution model, the number of messages exchanged, whether public keys are reused, and whether certificates or other protocol metadata are included. The object-size measurements in this experiment do not include these additional protocol components.

Private-key storage sizes are reported separately from transmitted data. In particular, the 64-byte ML-KEM value returned by `private_bytes_raw()` is the seed representation exposed by the installed API, not the full expanded decapsulation-key representation.

These measurements provide a cryptographic object-size comparison rather than a complete network-protocol bandwidth evaluation.

```
# Section 8.3 — Derive Standardized Object-Size Comparisons

MLKEM768_PUBLIC_KEY_STANDARD_BYTES = 1184
MLKEM768_CIPHERTEXT_STANDARD_BYTES = 1088
X25519_PUBLIC_KEY_STANDARD_BYTES = 32

size_comparison = {
    "mlkem_public_key_bytes": MLKEM768_PUBLIC_KEY_STANDARD_BYTES,
    "x25519_public_key_bytes": X25519_PUBLIC_KEY_STANDARD_BYTES,
    "mlkem_ciphertext_bytes": MLKEM768_CIPHERTEXT_STANDARD_BYTES,
    "mlkem_public_key_size_ratio": (
        MLKEM768_PUBLIC_KEY_STANDARD_BYTES
        / X25519_PUBLIC_KEY_STANDARD_BYTES
    ),
    "mlkem_ciphertext_vs_x25519_public_key_bytes": (
        MLKEM768_CIPHERTEXT_STANDARD_BYTES
        - X25519_PUBLIC_KEY_STANDARD_BYTES
    ),
}

print("Standardized object-size comparison")
print(f"ML-KEM-768 public key:
{size_comparison['mlkem_public_key_bytes']} bytes")
print(f"X25519 public key:
{size_comparison['x25519_public_key_bytes']} bytes")
print(f"ML-KEM-768 ciphertext:
{size_comparison['mlkem_ciphertext_bytes']} bytes")
print(
    "ML-KEM public-key size ratio: "
    f"{size_comparison['mlkem_public_key_size_ratio']:.2f}x"
)
print(
    "Ciphertext minus one X25519 public key: "
    f"{size_comparison['mlkem_ciphertext_vs_x25519_public_key_bytes']}
```

```
bytes"
)

Standardized object-size comparison
ML-KEM-768 public key: 1184 bytes
X25519 public key: 32 bytes
ML-KEM-768 ciphertext: 1088 bytes
ML-KEM public-key size ratio: 37.00x
Ciphertext minus one X25519 public key: 1056 bytes
```

9. Statistical Analysis and Comparative Evaluation

9.1 Objective

This section derives comparative statistics from the raw benchmark observations collected in Section 7. The analysis compares operation-level latency distributions, summarizes relative performance, and preserves the distinction between individual operations and complete key-establishment workflows.

9.2 Descriptive Statistics

For each operation, the analysis reports the sample count, mean, median, standard deviation, minimum, maximum, and 95th percentile latency.

The median is used as the primary measure of typical observed latency. The mean and standard deviation describe the central tendency and dispersion of the recorded samples, while the 95th percentile summarizes the upper portion of the observed latency distribution.

These statistics are descriptive. They do not, by themselves, establish statistical significance or guarantee performance under different hardware, operating systems, workloads, or cryptographic implementations.

9.3 Relative Latency

Relative latency ratios will be calculated for clearly paired operations. These ratios describe the observed execution-time relationship for the selected operations under the measured conditions.

The analysis will avoid comparing unlike operations as though they performed identical functions. In particular, ML-KEM encapsulation will not be treated as a direct semantic equivalent of X25519 shared-secret computation. Any comparison with the combined X25519 + HKDF-SHA256 operation will be explicitly labelled as an operation-level timing comparison rather than an equivalence claim.

9.4 Statistical Limitations

The observations are collected sequentially on a general-purpose operating system. Background processes, scheduling, CPU state, and other environmental effects can influence individual timings.

The experiment reports observed variability and percentiles but does not infer universal performance differences from a single machine. Additional independent runs would be needed to characterize between-run variability and improve confidence in the reproducibility of the findings.

```
# Section 9.2 — Comparative Latency Analysis

required_operations = {
    "mlkem768_key_generation",
    "mlkem768_encapsulation",
    "mlkem768_decapsulation",
    "x25519_key_generation",
    "x25519_shared_secret",
    "hkdf_sha256_derivation",
    "x25519_hkdf_combined_derivation",
}

assert required_operations.issubset(raw_benchmark_results.keys())

comparison_pairs = [
    (
        "Key generation",
        "mlkem768_key_generation",
        "x25519_key_generation",
    ),
    (
        "ML-KEM decapsulation vs X25519 agreement",
        "mlkem768_decapsulation",
        "x25519_shared_secret",
    ),
    (
        "ML-KEM encapsulation vs combined classical derivation",
        "mlkem768_encapsulation",
        "x25519_hkdf_combined_derivation",
    ),
]

comparison_rows = []

for comparison_name, mlkem_operation, classical_operation in
comparison_pairs:
    mlkem_median = float(
        np.median(raw_benchmark_results[mlkem_operation])
    )
    classical_median = float(
        np.median(raw_benchmark_results[classical_operation])
    )

    comparison_rows.append({
        "comparison": comparison_name,
```

```
        "mlkem_or_pqc_operation": mlkem_operation,
        "classical_operation": classical_operation,
        "pqc_median_ms": mlkem_median,
        "classical_median_ms": classical_median,
        "median_latency_ratio": mlkem_median / classical_median,
    })

comparison_df = pd.DataFrame(comparison_rows)

COMPARISON_RESULTS_PATH = PQC_ARTIFACT_DIR /
"pqc_classical_comparison.csv"
comparison_df.to_csv(COMPARISON_RESULTS_PATH, index=False)

display(comparison_df.round(6))
print(f"\nComparison table saved to: {COMPARISON_RESULTS_PATH}")
```

```
                                          comparison
mlkem_or_pqc_operation  \
0                                    Key generation
mlkem768_key_generation
1           ML-KEM decapsulation vs X25519 agreement
mlkem768_decapsulation
2  ML-KEM encapsulation vs combined classical der...
mlkem768_encapsulation

               classical_operation  pqc_median_ms  classical_median_ms
\
0            x25519_key_generation        0.37825                0.0892

1             x25519_shared_secret        0.14360                0.0768

2  x25519_hkdf_combined_derivation        0.09590                0.0879


   median_latency_ratio
0              4.240471
1              1.869792
2              1.091013


Comparison table saved to: C:\Projects\Aeges-Q\artifacts\pqc\
pqc_classical_comparison.csv
```

10. Conclusions, Limitations, and Future Work

10.1 Experimental Summary

This experiment implemented and evaluated ML-KEM-768 using the `cryptography` package and compared its operation-level performance with the existing AEGES-Q X25519 + HKDF-SHA256 implementation.

The ML-KEM-768 API passed an initial encapsulation-decapsulation smoke test and 100 repeated correctness trials. The classical implementation successfully established matching session keys between two independently generated X25519 key pairs.

The primary benchmark collected 500 recorded measurements for each of seven operations, producing 3,500 raw observations. The results include key-generation, encapsulation, decapsulation, shared-secret computation, HKDF derivation, and combined classical session-key derivation latency.

10.2 Performance Findings

In the measured environment, the median ML-KEM-768 key-generation latency was 0.37825 ms, compared with 0.08920 ms for X25519 key generation. ML-KEM-768 encapsulation had a median latency of 0.09590 ms, and decapsulation had a median latency of 0.14360 ms.

The existing X25519 + HKDF-SHA256 combined derivation operation had a median latency of 0.08790 ms. The measured ML-KEM encapsulation median was approximately 1.09 times this value, although the operations have different cryptographic semantics and boundaries.

These measurements characterize the tested software and hardware environment. They do not establish universal performance rankings or guarantee equivalent end-to-end protocol latency.

10.3 Object-Size Findings

The measured encoded ML-KEM-768 public key was 1,184 bytes, its encapsulation ciphertext was 1,088 bytes, and its shared secret was 32 bytes. The `private_bytes_raw()` API returned a 64-byte ML-KEM private-key seed representation.

The X25519 public-key encoding was 32 bytes, and the raw X25519 private-key encoding was also 32 bytes. The classical shared-secret output and derived session key were each 32 bytes.

Consequently, ML-KEM-768 has higher public-key and ciphertext data-size requirements than the X25519 public-key encoding. The effect on actual protocol bandwidth depends on the key-distribution model, message sequence, key reuse, and additional protocol metadata.

10.4 Correctness and Failure-Case Findings

The functional tests confirmed successful ML-KEM encapsulation-decapsulation consistency across 100 trials and successful two-party key derivation using the existing classical service.

The installed ML-KEM API rejected the tested invalid-length ciphertext with a `ValueError`. A same-length modified ciphertext returned a 32-byte shared secret different from the original. This observation does not establish ciphertext authentication, nor does it constitute a comprehensive test of malformed-input handling.

ML-KEM is a key-encapsulation mechanism, not an authenticated-encryption scheme. Applications must use appropriate cryptographic protocols and authenticated encryption to protect application data.

10.5 Limitations

The principal limitations of this experiment are:

1. **Single-machine evaluation:** The measurements were collected on one Windows 11 system with Python 3.13.7 and `cryptography` 50.0.1.
2. **Single primary benchmark run:** The recorded observations characterize one run and do not quantify between-run variability.
3. **Local operation measurements:** Network latency, concurrent session load, and full application integration were not measured.
4. **Implementation dependence:** Results depend on the Python API, underlying cryptographic backend, runtime, operating system, and hardware.
5. **Limited failure-case coverage:** The malformed-ciphertext tests cover selected inputs and do not constitute exhaustive cryptographic validation.
6. **Protocol-level differences:** ML-KEM encapsulation and X25519 agreement have different interfaces, so operation-level latency comparisons must be interpreted carefully.
7. **No end-to-end security evaluation:** This experiment does not prove security, assess side-channel resistance, or evaluate every deployment threat.

10.6 Future Work

Future experiments should investigate:

- Independent benchmark reruns and confidence intervals for latency estimates.
- Throughput and latency under increasing concurrent session-establishment loads.
- Full protocol message sizes, including certificates, framing, and other metadata where applicable.
- Integration of ML-KEM-768 into the AEGES-Q session-establishment architecture.
- Authenticated-encryption integration using the existing AES-256-GCM service.
- Interoperability testing against an independent standards-conformant ML-KEM implementation.
- A carefully designed hybrid key-establishment experiment if hybrid classical and post-quantum protection becomes an explicit project objective.

10.7 Conclusion

The experiment establishes a functional ML-KEM-768 implementation and a reproducible initial benchmark alongside the existing AEGES-Q classical key-establishment baseline. The results provide an initial characterization of computation time and encoded object sizes.

The findings support continued investigation of post-quantum cryptography within AEGES-Q. Further independent runs, protocol-level testing, and system integration measurements are required before making broader claims about deployment performance or security.

10.8 Reproducibility and Research-Readiness Checklist

Before using the results in a research paper or final project report, verify the following:

- ☒ ML-KEM-768 API availability confirmed in the existing environment.
- ☒ Initial encapsulation-decapsulation smoke test passed.
- ☒ 100 repeated ML-KEM correctness trials passed.

- ☒ Existing X25519 + HKDF-SHA256 implementation validated without source modifications.
- ☒ Classical public-key serialization and malformed-length checks passed.
- ☒ Selected ML-KEM malformed-ciphertext behaviours recorded.
- ☒ Benchmark configuration saved before the main benchmark.
- ☒ Seven cryptographic operations measured with 500 recorded samples each.
- ☒ Raw timing observations and summary statistics saved.
- ☒ Cryptographic object sizes measured and saved.
- ☒ Comparative latency table generated.
- ☐ Independent benchmark rerun completed.
- ☐ Full project test suite rerun after notebook work.
- ☐ Interoperability against an independent ML-KEM implementation evaluated.
- ☐ Protocol-level or concurrent-load performance evaluated if required by the final research claims.

The completed items establish the initial implementation and measurement baseline. The unchecked items remain future validation tasks and must not be represented as completed experiments.

```
# Section 10.3 — Final Artifact Audit

expected_artifacts = [
    PQC_ARTIFACT_DIR / "pqc_environment_metadata.json",
    PQC_ARTIFACT_DIR / "pqc_benchmark_config.json",
    PQC_ARTIFACT_DIR / "pqc_raw_benchmark_results.csv",
    PQC_ARTIFACT_DIR / "pqc_benchmark_summary.csv",
    PQC_ARTIFACT_DIR / "pqc_cryptographic_object_sizes.csv",
    PQC_ARTIFACT_DIR / "pqc_classical_comparison.csv",
]

artifact_audit_rows = []

for artifact_path in expected_artifacts:
    exists = artifact_path.is_file()
    artifact_audit_rows.append({
        "artifact": str(artifact_path.relative_to(PROJECT_ROOT)),
        "exists": exists,
        "size_bytes": artifact_path.stat().st_size if exists else
None,
    })

artifact_audit_df = pd.DataFrame(artifact_audit_rows)
display(artifact_audit_df)

missing_artifacts = artifact_audit_df.loc[
    ~artifact_audit_df["exists"], "artifact"
].tolist()
```

```python
if missing_artifacts:
    print("\nMissing artifacts:")
    for artifact in missing_artifacts:
        print("-", artifact)
else:
    print("\nAll expected artifacts are present.")

print(f"\nArtifact directory: {PQC_ARTIFACT_DIR}")
```

```
                                          artifact  exists
size_bytes
0       artifacts\pqc\pqc_environment_metadata.json    True
756
1           artifacts\pqc\pqc_benchmark_config.json    True
668
2       artifacts\pqc\pqc_raw_benchmark_results.csv    True
362830
3           artifacts\pqc\pqc_benchmark_summary.csv    True
772
4  artifacts\pqc\pqc_cryptographic_object_sizes.csv    True
571
5        artifacts\pqc\pqc_classical_comparison.csv    True
466


All expected artifacts are present.

Artifact directory: C:\Projects\Aeges-Q\artifacts\pqc
```